

Project Progress Report: Week 1

Project Title: GUI based Car Racing Game using java

Task Domain: Requirements Collection, Component Study, & Game Design Architecture

1. Project Requirements Collection

The core operational, structural, and asset-based requirements for the car racing game were compiled to establish solid development boundaries before beginning implementation:

- **Application Window Frame:** A restricted, non-resizable canvas measuring exactly 600 pixels (Width) X 700 pixels (Height).
- **Target Frame Rate Engine:** Controlled refresh rate cycles targeting a 20ms delay execution loop using a swing timer, resulting in approximately 50 frames per second (FPS) for smooth animations.
- **Control Schema:** Direct input mapping utilizing keyboard components. Specifically, ENTER to initiate gameplay from the menu, LEFT and RIGHT arrow keys for horizontal player navigation, and R to execute runtime game resets.
- **Audio Asset Directory:** Isolation of four core .wav external files required to manage audio feedback:
 1. start.wav: Looping menu tracking track.
 2. run.wav: Looping active gameplay ambient engine sound.
 3. score.wav: Incident audio notification triggered when successfully bypassing obstacles.
 4. crash.wav: Definitive single-play event audio clip triggered on collision.

2. Technical Evaluation of Java Swing Components

A deep architectural study of the javax.swing and java.awt packages was conducted to map structural dependencies to game features:

Window Management (JFrame)

The structural frame of the application. It acts as the primary top-level container, controlling frame borders, absolute window scale dimensions, exit behaviors upon window closure (JFrame.EXIT_ON_CLOSE), and acts as the structural host for the game panel interface.

Graphic Rendering Surface (JPanel & paintComponent)

The custom rendering canvas for the game. By overriding the native paintComponent(Graphics g) method, it grants direct access to the Graphics 2D raster toolset. This allows procedural

geometric shapes to form the backdrop environments, lane lines, structural vehicles, and overlay text headers directly onto the screen matrix.

Frame State Automation Loop (javax.swing.Timer)

An engine that fires automated action events at fixed intervals (20ms). It acts directly within the Java Event Dispatch Thread (EDT). This approach is mandatory for programmatic GUI games; using a standard CPU execution loop (while(true)) blocks the EDT thread, freezing screens instantly.

Input Event Listeners (KeyListener)

An asynchronous hardware checking class mapped to interface focus tracking. It overrides key intercept models (keyPressed) to listen for specific keycode constants (KeyEvent.VK_ENTER, KeyEvent.VK_LEFT, KeyEvent.VK_RIGHT, KeyEvent.VK_R), transferring raw keystrokes directly into instant in-game positional shifts or system state transitions.

3. Game Design Structure & Grid Coordinates Analysis

The execution layout was mathematically mapped out onto an inverted cartesian coordinate grid, where the origin point (0,0) resides explicitly at the **top-left corner** of the screen window.

The Illusion of Speed (Vertical Scrolling Physics)

To eliminate the programmatic complexity of an infinite forward grid, the player's car remains completely locked along the vertical Y-axis (playerY = 500). The structural visual impression of high-speed racing is simulated dynamically: the background divider road coordinates (roadY) and obstacles (enemyY) continuously translate downward along the vertical Y-axis (+7 pixels per frame refresh step).

Playable Lane Restrictions

The road bounds span centrally from X = 150 to X = 450 pixels. To ensure player safety logic behaves within standard highway boundaries, hard boundaries are coded directly into the movement mechanics:

Boundary Equation: $170 \leq \text{playerX} \leq 370$

This structural mathematical restraint prevents the player car model from driving off the road into the grass areas.

Procedural Generation System

Enemy vehicles spawn entirely off-screen along a negative spatial baseline (enemyY = -100) to guarantee clean entry translations into the viewport. Upon reaching the boundary edge (enemyY > 700), the tracking application automatically cleans up the object tracking instance, adds +1 point to the user score metric, and re-initializes the enemy placement using a pseudo-random seed distribution:

Random X-Spawn Location: $X = 170 + \text{random.nextInt}(220)$

4. Development Summary Roadmap

With requirements cataloged, components reviewed, and coordinate math finalized, the architectural phase of Week 1 is complete. The project is systematically staged to transition into Week 2 for NetBeans workspace creation, primary project container setup, and system initialization routines.